

Work Breakdown Structure for BacTools: Automated Software Catalog Creator for Bacterial Data Analysis

Week 1: Set Up GitHub Repository and write high level project description / documentation

- ☒ Initialize GitHub repository
- ☒ Push SRS.md and datasets.md to remote repository
- ☒ Push WBS document to repository
- ☒ List tasks for software catalog creation
- ☒ List tasks for keyword management and validation
- ☒ Break project into modules

Week 2: Source and Preprocess Metadata, Implement Auxiliary Scripts, Validate Example Query

- ☐ Restructure how GitHub tokens and API requests are made by making them user generated inputs
 - ☐ Sub actions
 - ☐ Implement requests for GitHub tokens and storage
 - ☐ Update API requests based on user requests
 - ☐ Tasks
 - ☐ Description: Modify initialization scripts (e.g., config.R, main.py) to prompt users to input their GitHub API token securely (e.g., via environment variable or input).
 - ☐ Deliverable: Updated script supporting user-driven token input.
 - ☐ Completion Criteria: Script executes and user can supply their token at runtime; token is not hard-coded or exposed in the source code.
- ☐ Double check functions to query and read tool metadata (JSON/XML/CSV format)
- ☐ Search CRAN, Bioconductor, and GitHub using relevant bacterial analysis keywords
 - ☐ Sub Actions
 - ☐ Expand API querying logic to fetch README files.
 - ☐ Text mine README content for keyword matches.
 - ☐ Tasks
 - ☐ Description: Extend search_github_tools to programmatically fetch the README file for each repository found via API.

- ☐ Deliverable: Code capable of retrieving README text from GitHub API for given repositories.
- ☐ Completion Criteria: For every repository in the result set, README content is fetched and temporarily stored for secondary processing.
- ☐ Description: Implement function to scan README for bacterial keywords (bacteria, genomics, AMR, etc.) and flag repositories that meet criteria.
- ☐ Deliverable: Keyword matching module and flagging system.
- ☐ Completion Criteria: Output includes identification of which repositories match based on README content; validated on test repositories.
- ☐ Perform time checks on initial API results
- ☐ Improve metadata cleaning
 - ☐ Sub actions:
 - ☐ Implement null value replacement logic.
 - ☐ Standardize missing field handling.
 - ☐ Harmonize metadata fields across repositories
 - ☐ Automate duplicate detection and removal.
 - ☐ Tasks:
 - ☐ Description: In the cleaning script (tool_list_to_df), replace NULL and missing values with empty strings or standardized placeholders for all tool metadata fields.
 - ☐ Deliverable: Updated cleaning script with robust null/missing field management.
 - ☐ Completion Criteria: The function outputs data frames without NULL values; all entries have valid fields (may be empty if data is absent).
 - ☐ Description: Write logic to identify duplicate tools based on DOI, or other identifiers, and merge or drop duplicates.
 - ☐ Deliverable: Automated duplicate checking module in pipeline.
 - ☐ Completion Criteria: Output data contains only unique tool entries; tested on sample data, reviewed, and merged.
 - ☐ Description: After cleaning, produce a report summarizing number of fields replaced, duplicates found and removed, and any problematic entries for manual review.
 - ☐ Deliverable: Automated generation of cleaning/validation log.
 - ☐ Completion Criteria: Log is generated and attached to each cleaning run; any anomalies are flagged for review.
- ☐ Classify tools by tags and categories (are there categories I can add?)
- ☐ Decide whether website based tools that are not in (CRAN, Bioconductor, and GitHub) like NIAID-funded Bacterial and Viral Bioinformatics Resource Center should be included
- ☐ Simplify helper functions for field standardization, duplicate detection, and tag assignment
- ☐ Develop scripts for summary statistics and automated validation checks

- ☐ Use a single keyword to test pipeline functionality
- ☐ Identify and troubleshoot edge cases

Week 3: Perform Exploratory Metadata Analysis and Finalize Catalog Pipeline Architecture

- ☐ Visualize tool category distribution and source platform statistics
- ☐ Compute basic stats (tools per category, average stars, most frequent languages)
- ☐ Identify common and outlier tool attributes
- ☐ Clean up data pipeline for automated querying, extraction, cleaning, and export
- ☐ Develop, test, and create pull requests for assigned features
- ☐ Review and provide feedback on pull requests
- ☐ Merge approved pull requests into main branch
- ☐ Finalize core modules for API integration, de-duplication, and metadata normalization

Week 4: Update Design and Documentation, Create User Tutorial and Validate Pipeline on Full Dataset

- ☐ Update WBS, SRS.md, and datasets.md to reflect current project status
- ☐ Update README.md with usage instructions and overview
- ☐ Update README.md with tutorial links and example results
- ☐ Document pipeline modules and functionality in code comments
- ☐ Update datasets.md with detailed description of example metadata and justification
- ☐ Create a Jupyter Notebook or Markdown tutorial for BacTools usage
- ☐ Demonstrate catalog creation, filtering, and export in tutorial materials
- ☐ Run pipeline on complete metadata from all repositories
- ☐ Troubleshoot performance bottlenecks and resolve metadata consistency issues
- ☐ Update tutorial to reflect results from full-scale dataset
- ☐ Finalize design documents, README.md, and prepare for demonstration